```cpp
#include <Arduino.h>
#include <math.h>
#include "driver/pcnt.h"

// ========================================================
// ENCODER PINS
// ========================================================

#define LEFT_A   32
#define LEFT_B   33

#define RIGHT_A  25
#define RIGHT_B  26

// ========================================================
// IMU PINS
// ========================================================

HardwareSerial IMU(2);

#define IMU_RX 16
#define IMU_TX 17

// ========================================================
// ENCODER AND WHEEL SETTINGS
// ========================================================

const long PPR = 2500;

const float WHEEL_RADIUS = 0.05f;

const float WHEEL_CIRCUMFERENCE =
  2.0f * PI * WHEEL_RADIUS;

const float DISTANCE_PER_PULSE =
  WHEEL_CIRCUMFERENCE / PPR;

// ========================================================
// PCNT SETTINGS
// ========================================================

#define LEFT_PCNT_UNIT  PCNT_UNIT_0
#define RIGHT_PCNT_UNIT PCNT_UNIT_1
```

```
// ========================================================
// ENCODER DISTANCE VARIABLES
// ========================================================

float leftDistance = 0.0f;
float rightDistance = 0.0f;

float encoderDistance = 0.0f;

// Movement detected by encoders but not yet accepted
float pendingEncoderDistance = 0.0f;

// ========================================================
// IMU ACCELERATION VARIABLES
// ========================================================

float measuredX = 0.0f;
float measuredY = 0.0f;
float measuredZ = 0.0f;

// One combined acceleration value
float acceleration = 0.0f;

// Filtered acceleration
float filteredAcceleration = 0.0f;

// ========================================================
// IMU VELOCITY AND DISTANCE
// ========================================================

float imuVelocity = 0.0f;
float imuDistance = 0.0f;

// ========================================================
// FUSED DISTANCE
// ========================================================

float fusedDistance = 0.0f;

// ========================================================
// TIMING
// ========================================================
```

```
const unsigned long SAMPLE_INTERVAL = 500;

unsigned long previousSampleTime = 0;

// ======================================================
// THRESHOLDS
// ======================================================

// Requested acceleration threshold
const float ACCELERATION_THRESHOLD = 1.0f;

// Small velocity threshold for encoder gating
const float VELOCITY_THRESHOLD = 0.05f;

// Small acceleration noise
const float ACCELERATION_DEADBAND = 0.08f;

// ======================================================
// SCALAR KALMAN FILTER FOR ACCELERATION
// ======================================================

class KalmanFilter {
  private:
    float estimate = 0.0f;
    float estimateError = 1.0f;

    float processNoise;
    float measurementNoise;

  public:
    KalmanFilter(float q, float r) {
      processNoise = q;
      measurementNoise = r;
    }

    float update(float measurement) {
      // Prediction
      estimateError += processNoise;

      // Kalman gain
      float kalmanGain =
        estimateError /
```

```
        (estimateError + measurementNoise);

      // Correction
      estimate =
        estimate +
        kalmanGain * (measurement - estimate);

      // Update error
      estimateError =
        (1.0f - kalmanGain) * estimateError;

      return estimate;
    }
};

// Scalar filter because we want one acceleration value
KalmanFilter accelerationKalman(0.05f, 0.5f);

// ========================================================
// KALMAN FILTER FOR ENCODER-PRIMARY DISTANCE FUSION
// ========================================================

class DistanceKalman {
  private:
    float estimate = 0.0f;
    float estimateError = 0.1f;

    // Small process noise
    const float processNoise = 0.01f;

    // High IMU measurement noise.
    // This keeps encoder as the primary source.
    const float imuMeasurementNoise = 4.0f;

  public:
    float update(
      float encoderMeasurement,
      float imuMeasurement
    ) {
      // Encoder is the primary prediction
      estimate = encoderMeasurement;

      // Increase prediction uncertainty
```

```
      estimateError += processNoise;

      // Kalman gain
      float kalmanGain =
        estimateError /
        (estimateError + imuMeasurementNoise);

      // IMU makes only a small correction
      estimate =
        estimate +
        kalmanGain *
        (imuMeasurement - estimate);

      // Update error
      estimateError =
        (1.0f - kalmanGain) * estimateError;

      if (estimate < 0.0f) {
        estimate = 0.0f;
      }

      return estimate;
    }
};

DistanceKalman distanceKalman;

// ========================================================
// ACCELERATION DEADBAND
// ========================================================

float applyDeadband(float value) {
  if (fabs(value) < ACCELERATION_DEADBAND) {
    return 0.0f;
  }

  return value;
}

// ========================================================
// LEFT ENCODER SETUP
// ========================================================
```

```
void setupLeftEncoder() {
  pcnt_config_t config = {};

  config.pulse_gpio_num = LEFT_A;
  config.ctrl_gpio_num = LEFT_B;

  config.unit = LEFT_PCNT_UNIT;
  config.channel = PCNT_CHANNEL_0;

  // 1X counting: rising edge of Channel A only
  config.pos_mode = PCNT_COUNT_INC;
  config.neg_mode = PCNT_COUNT_DIS;

  config.lctrl_mode = PCNT_MODE_KEEP;
  config.hctrl_mode = PCNT_MODE_KEEP;

  config.counter_h_lim = 32767;
  config.counter_l_lim = 0;

  pcnt_unit_config(&config);

  pcnt_counter_pause(LEFT_PCNT_UNIT);
  pcnt_counter_clear(LEFT_PCNT_UNIT);
  pcnt_counter_resume(LEFT_PCNT_UNIT);
}

// ========================================================
// RIGHT ENCODER SETUP
// ========================================================

void setupRightEncoder() {
  pcnt_config_t config = {};

  config.pulse_gpio_num = RIGHT_A;
  config.ctrl_gpio_num = RIGHT_B;

  config.unit = RIGHT_PCNT_UNIT;
  config.channel = PCNT_CHANNEL_0;

  // 1X counting: rising edge of Channel A only
  config.pos_mode = PCNT_COUNT_INC;
  config.neg_mode = PCNT_COUNT_DIS;
```

```
  config.lctrl_mode = PCNT_MODE_KEEP;
  config.hctrl_mode = PCNT_MODE_KEEP;

  config.counter_h_lim = 32767;
  config.counter_l_lim = 0;

  pcnt_unit_config(&config);

  pcnt_counter_pause(RIGHT_PCNT_UNIT);
  pcnt_counter_clear(RIGHT_PCNT_UNIT);
  pcnt_counter_resume(RIGHT_PCNT_UNIT);
}

// ======================================================
// COLLECT ENCODER MOVEMENT
// ======================================================

// This part is kept the same.
// It stores encoder movement temporarily.

void collectEncoderMovement() {
  int16_t leftPulses = 0;
  int16_t rightPulses = 0;

  pcnt_get_counter_value(
    LEFT_PCNT_UNIT,
    &leftPulses
  );

  pcnt_get_counter_value(
    RIGHT_PCNT_UNIT,
    &rightPulses
  );

  // Clear counters after reading
  pcnt_counter_clear(LEFT_PCNT_UNIT);
  pcnt_counter_clear(RIGHT_PCNT_UNIT);

  float leftMovement =
    leftPulses * DISTANCE_PER_PULSE;

  float rightMovement =
    rightPulses * DISTANCE_PER_PULSE;
```

```
  float movement =
    (leftMovement + rightMovement) / 2.0f;

  // Store movement temporarily
  pendingEncoderDistance += movement;
}

// ========================================================
// READ JY901S IMU ACCELERATION
// ========================================================

void readIMU() {
  static uint8_t packet[11];
  static int packetIndex = 0;

  while (IMU.available()) {
    uint8_t incomingByte = IMU.read();

    // Find packet header
    if (packetIndex == 0) {
      if (incomingByte == 0x55) {
        packet[packetIndex++] = incomingByte;
      }

      continue;
    }

    packet[packetIndex++] = incomingByte;

    if (packetIndex < 11) {
      continue;
    }

    // Packet completed
    packetIndex = 0;

    // 0x51 = acceleration packet
    if (packet[1] != 0x51) {
      continue;
    }

    // Check checksum
```

```
uint8_t checksum = 0;

for (int i = 0; i < 10; i++) {
  checksum += packet[i];
}

if (checksum != packet[10]) {
  continue;
}

// Extract raw acceleration
int16_t rawX =
  (int16_t)(((uint16_t)packet[3] << 8) | packet[2]);

int16_t rawY =
  (int16_t)(((uint16_t)packet[5] << 8) | packet[4]);

int16_t rawZ =
  (int16_t)(((uint16_t)packet[7] << 8) | packet[6]);

// Convert to m/s²
// JY901S range = ±16 g

measuredX =
  ((float)rawX / 32768.0f) * 16.0f * 9.80665f;

measuredY =
  ((float)rawY / 32768.0f) * 16.0f * 9.80665f;

measuredZ =
  ((float)rawZ / 32768.0f) * 16.0f * 9.80665f;

// ------------------------------------------------
// Combine 3 axes into one acceleration magnitude
// ------------------------------------------------

float totalAcceleration =
  sqrt(
    measuredX * measuredX +
    measuredY * measuredY +
    measuredZ * measuredZ
  );
```

```
    // ------------------------------------------------
    // Remove gravity
    //
    // At rest, total acceleration is approximately
    // 9.80665 m/s² because of gravity.
    // ------------------------------------------------

    float dynamicAcceleration =
      fabs(totalAcceleration - 9.80665f);

    // Ignore small noise
    dynamicAcceleration =
      applyDeadband(dynamicAcceleration);

    // Apply scalar Kalman filter
    filteredAcceleration =
      accelerationKalman.update(dynamicAcceleration);

    // Ignore negative values
    acceleration =
      fabs(filteredAcceleration);
  }
}

// ======================================================
// UPDATE IMU VELOCITY AND DISTANCE
// ======================================================

void updateIMUVelocityAndDistance(float dt) {
  /*
     Acceleration is treated as a magnitude.
     Therefore, forward and backward movement are
     both represented as positive speed/distance.
  */

  float usableAcceleration = acceleration;

  // Only use acceleration above the requested threshold
  if (usableAcceleration < ACCELERATION_THRESHOLD) {
    usableAcceleration = 0.0f;
  }

  // ------------------------------------------------
```

```
  // Velocity equation:
  //
  // v = u + at
  // -----------------------------------------------------

  float initialVelocity = imuVelocity;

  if (usableAcceleration > 0.0f) {
    imuVelocity =
      initialVelocity +
      usableAcceleration * dt;
  } else {
    /*
       When acceleration becomes low, gradually reduce
       velocity so that the robot can eventually become
       stationary instead of keeping old velocity forever.
    */

    imuVelocity *= 0.5f;

    if (imuVelocity < VELOCITY_THRESHOLD) {
      imuVelocity = 0.0f;
    }
  }

  // -----------------------------------------------------
  // Distance equation:
  //
  // s = ut + 0.5at²
  // -----------------------------------------------------

  float displacement =
    initialVelocity * dt +
    0.5f * usableAcceleration * dt * dt;

  // Ignore negative distance
  if (displacement > 0.0f) {
    imuDistance += displacement;
  }
}

// =======================================================
// ACCEPT OR REJECT ENCODER MOVEMENT USING VELOCITY
```

```
// ========================================================

void updateEncoderDistanceUsingVelocity() {
  /*
     Encoder movement is accepted only when the IMU
     velocity is greater than the threshold.
  */

  if (fabs(imuVelocity) >= VELOCITY_THRESHOLD) {
    encoderDistance += pendingEncoderDistance;

    // These variables are retained from your original code
    leftDistance += pendingEncoderDistance / 2.0f;
    rightDistance += pendingEncoderDistance / 2.0f;
  }

  // Clear pending movement after decision
  pendingEncoderDistance = 0.0f;
}

// ========================================================
// SETUP
// ========================================================

void setup() {
  Serial.begin(115200);

  // Encoder pins
  pinMode(LEFT_A, INPUT_PULLUP);
  pinMode(LEFT_B, INPUT_PULLUP);

  pinMode(RIGHT_A, INPUT_PULLUP);
  pinMode(RIGHT_B, INPUT_PULLUP);

  // Setup PCNT
  setupLeftEncoder();
  setupRightEncoder();

  // Start JY901S
  IMU.begin(
    9600,
    SERIAL_8N1,
    IMU_RX,
```

```
    IMU_TX
  );

  previousSampleTime = millis();

  Serial.println();
  Serial.println("========================================");
  Serial.println("ENCODER PRIMARY + IMU KALMAN FUSION");
  Serial.println("========================================");
  Serial.println("Encoder: PCNT 1X Counting");
  Serial.println("IMU: Combined Acceleration + Kalman Filter");
  Serial.println("Acceleration Threshold: 1.0 m/s²");
  Serial.println("Sampling Interval: 500 ms");
  Serial.println();
}

// ======================================================
// MAIN LOOP
// ======================================================

void loop() {
  // Read IMU continuously
  readIMU();

  // Collect encoder pulses continuously
  collectEncoderMovement();

  unsigned long currentMillis = millis();

  if (currentMillis - previousSampleTime >= SAMPLE_INTERVAL) {
    previousSampleTime = currentMillis;

    float dt = SAMPLE_INTERVAL / 1000.0f;

    // 1. Calculate IMU velocity and distance
    updateIMUVelocityAndDistance(dt);

    // 2. Accept encoder movement only if velocity exists
    updateEncoderDistanceUsingVelocity();

    // 3. Fuse encoder-primary distance with IMU distance
    fusedDistance =
      distanceKalman.update(
```

```
        encoderDistance,
        imuDistance
      );

    // 4. Print required values
    Serial.println("-----------------------------------------");

    Serial.print("Acceleration: ");
    Serial.print(acceleration, 3);
    Serial.println(" m/s²");

    Serial.print("Velocity: ");
    Serial.print(imuVelocity, 3);
    Serial.println(" m/s");

    Serial.print("IMU Distance: ");
    Serial.print(imuDistance, 3);
    Serial.println(" m");

    Serial.print("Encoder Distance: ");
    Serial.print(encoderDistance, 3);
    Serial.println(" m");

    Serial.print("Fused Distance: ");
    Serial.print(fusedDistance, 3);
    Serial.println(" m");
  }
}
```